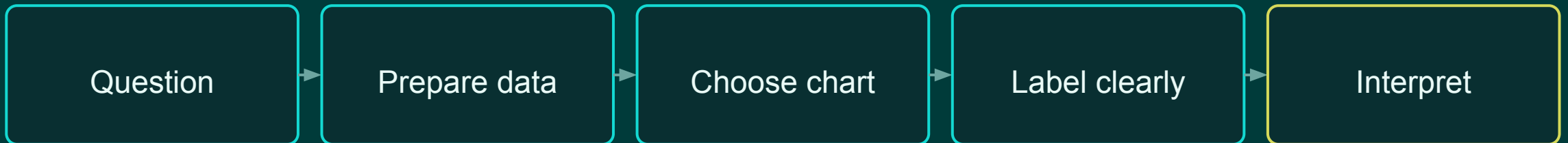


Static Visualization with pandas and Matplotlib

A one-block notebook lesson: turn pandas answers into clear charts

Appendix D.1

Visualize the answer, not the whole dataframe.



Mental model: a chart is a visual answer to a specific question.

Block Plan: 30 Minutes Teach + 30 Minutes Practice

The lesson moves from chart purpose to a notebook mini-report

Instruction

30 minutes

Questions

pandas plot

Matplotlib labels

Chart types

Honest choices

Guided Exercise

30 minutes

Load data

Source bars

Volume bars

Histogram

Weekly line

Applied Challenge

Static overview

4 charts

plus Markdown interpretation

finance 

logistics 

maintenance 

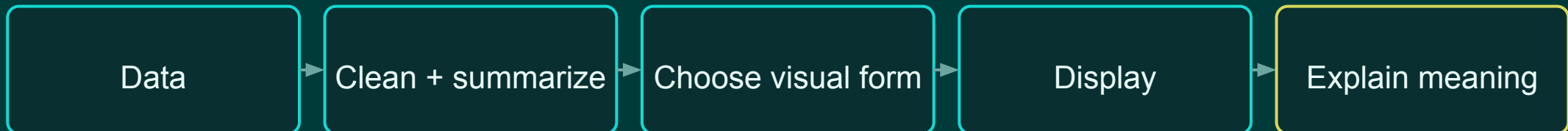
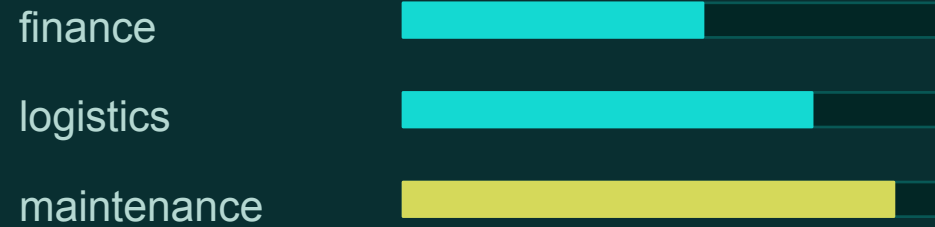
Why Visualize Data?

Tables contain answers; charts help people recognize comparisons and patterns

source	
finance	482000
logistics	615000
maintenance	734000

Same answer, easier scan

bar length carries the comparison



Visualization does not replace analysis. It communicates the result of an analysis.

Visualization Begins with a Question

Ask what you need to know before choosing the chart

Weak starting point

“What chart can I make?”

This often leads to plotting every column and hoping a pattern appears.

Plot everything → hope meaning
appears

Better starting point

“What question am I trying to answer?”

This guides the calculation, chart type, labels, and interpretation.

Question → calculation → chart

Question-to-Chart Decision Guide

The chart type should follow the kind of comparison

Compare categories

Bar chart

records by source
average errors by source

Change over time

Line chart

daily or weekly volume

Distribution

Histogram

completion percentages

Two numeric values

Scatter plot

errors vs completion

Distribution by category

Box plot

completion by source

Practical rule: the same dataframe can support many charts, but not every chart answers the same question.

Visualization Is Part of the Data Pipeline

A chart normally comes after inspection, cleaning, filtering, and aggregation



```
# Weak: no clear analytical question
records.plot()
```

```
# Better: prepare the answer first
records_by_source = (
    records["source"]
    .value_counts()
    .sort_values()
)

records_by_source.plot(kind="barh")
```

Prepare answer → plot answer

Notebook Setup

Use pandas for data and Matplotlib for static chart display

```
import pandas as pd
import matplotlib.pyplot as plt

records = pd.read_csv(
    "block_42_equipment_activity.csv"
)

records["processed_date"] = pd.to_datetime(
    records["processed_date"]
)
```

Notebook habits

- Put one major chart per cell
- Use `plt.show()` to display intentionally
- Parse dates before time charts
- Inspect `head()` and `info()` before plotting
- Keep code, chart, and interpretation together

Static charts appear beneath the cell that creates them.

First Chart: Counts by Source

`value_counts()` creates the answer; `barh` displays it

```
records_by_source = (  
    records["source"]  
    .value_counts()  
    .sort_values()  
)  
  
records_by_source.plot(  
    kind="barh"  
)  
  
plt.show()
```

Series mental model

index = labels, values = bar lengths

finance



logistics



maintenance



`records_by_source` is a Series: source names are the index; counts are the values.

pandas Starts the Chart; Matplotlib Finishes It

Capture the axes object, then add labels and presentation details

```
axes = records_by_source.plot(  
    kind="barh",  
    figsize=(8, 5),  
)  
  
axes.set_title("Records Received by Source")  
axes.set_xlabel("Number of Records")  
axes.set_ylabel("Source System")  
  
plt.show()
```

pandas Prepares and starts the chart from Series/DataFrame data.

Matplotlib Controls titles, labels, figure size, layout, and saved images.

A chart should not require the viewer to inspect the code to understand it.

Counts Versus Totals

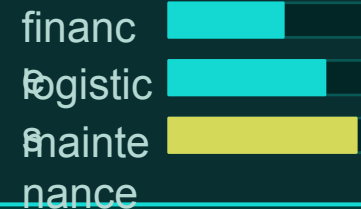
Two similar-looking charts can answer different questions

How many rows per source?

```
row_counts = (
    records["source"]
    .value_counts()
    .sort_values()
)
```

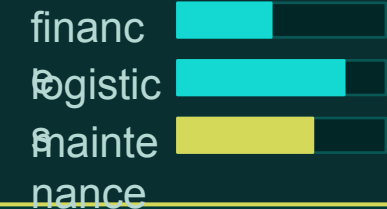
Rows per source

value_counts()



Volume by source

groupby().sum()



How much volume per source?

```
source_totals = (
    records.groupby("source")["record_count"]
    .sum()
    .sort_values()
)
```

Chart correctness depends on the calculation behind the chart.

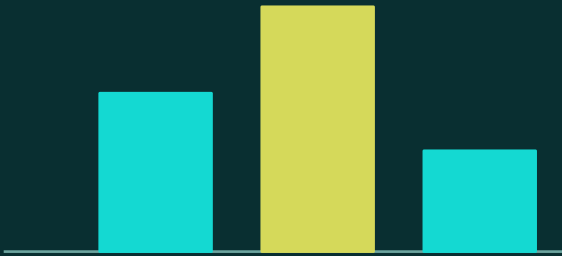
Most rows and greatest total record_count are not necessarily the same result.

Bar Charts: Category Comparisons

Use bars when the question compares values across categories

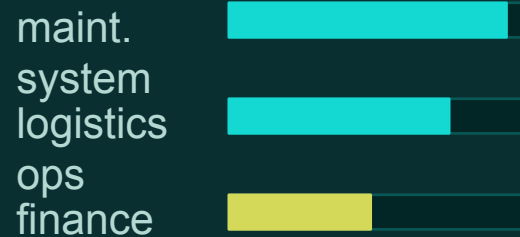
Vertical bars

short labels can work well



Horizontal bars

long labels are easier to read



Sort first

Sorting bars helps the viewer see ranking immediately.

```
average_errors = (  
    average_errors  
    .sort_values()  
)
```

Bar chart rule: label the metric clearly — row count, total volume, average, or something else.

Line Charts: Ordered Time

A line implies meaningful order or continuity

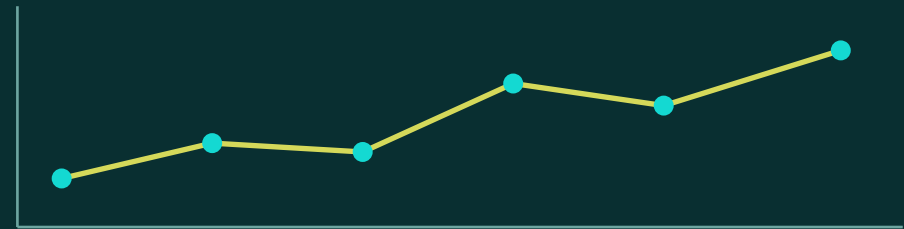
```
daily_volume = (
    records.groupby("processed_date")["record_count"]
    .sum()
    .sort_index()
)

axes = daily_volume.plot(
    kind="line",
    figsize=(10, 5),
)

axes.grid(True)
plt.show()
```

Daily processing volume

date order matters



Avoid this

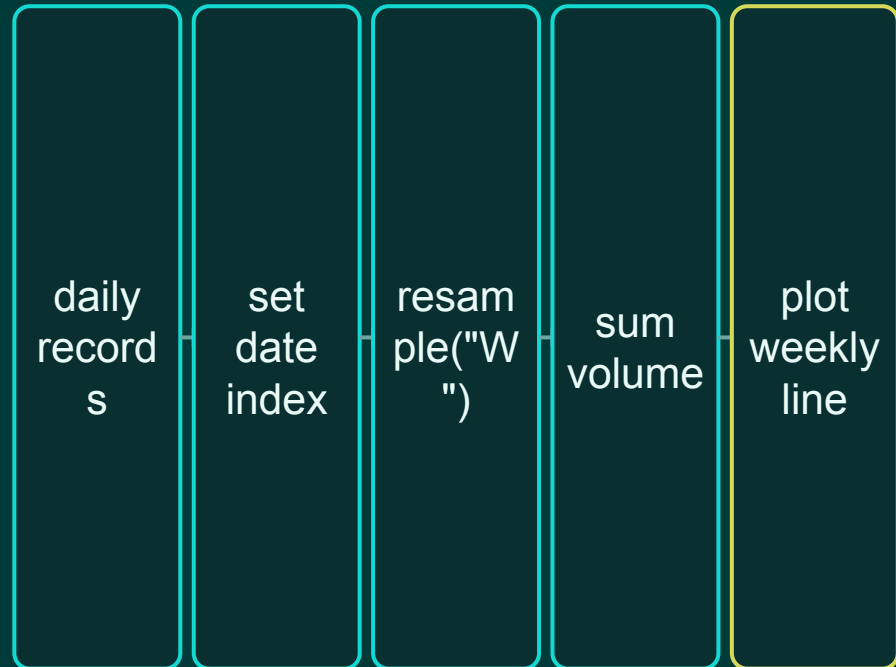
Do not connect unordered categories like
finance → logistics → maintenance with a
line.

Use `.sort_index()` so dates appear
chronologically.

Resampling Dates

Time-based data can be summarized into meaningful intervals

```
weekly_volume = (  
    records.set_index("processed_date")  
    .resample("W")["record_count"]  
    .sum()  
)  
  
axes = weekly_volume.plot(  
    kind="line",  
    figsize=(10, 5),  
    marker="o",  
)  
  
axes.set_title("Weekly Processing Volume")  
plt.show()
```



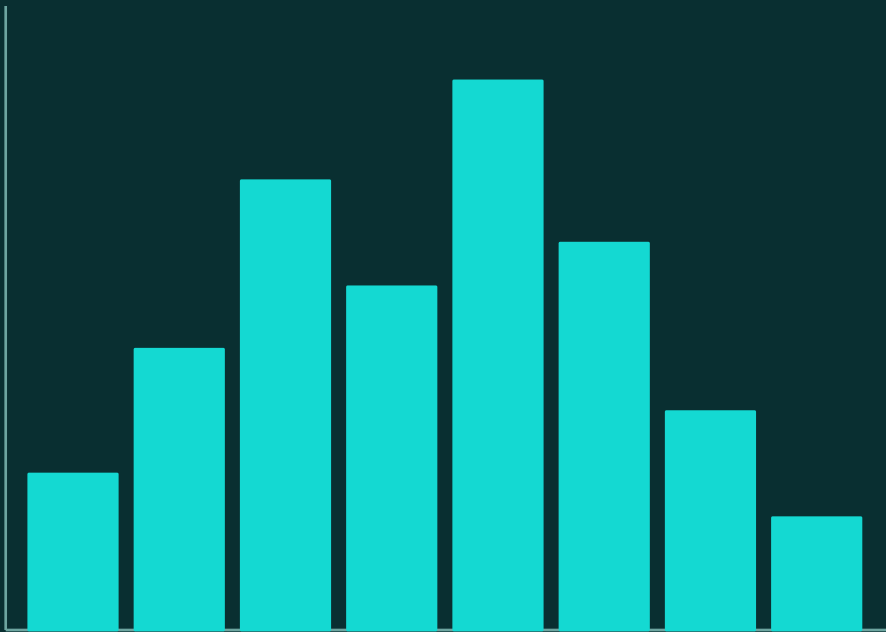
Resampling changes the time grain, not the underlying historical facts.

Histograms: Numeric Distribution

A histogram groups numeric values into intervals called bins

Completion distribution

How are completion percentages spread?



```
axes = records["completion"].plot(  
    kind="hist",  
    bins=10,  
    figsize=(8, 5),  
    edgecolor="black",  
)  
  
axes.set_title(  
    "Distribution of Completion Percentages"  
)  
axes.set_xlabel("Completion Percentage")  
axes.set_ylabel("Number of Records")  
  
plt.show()
```

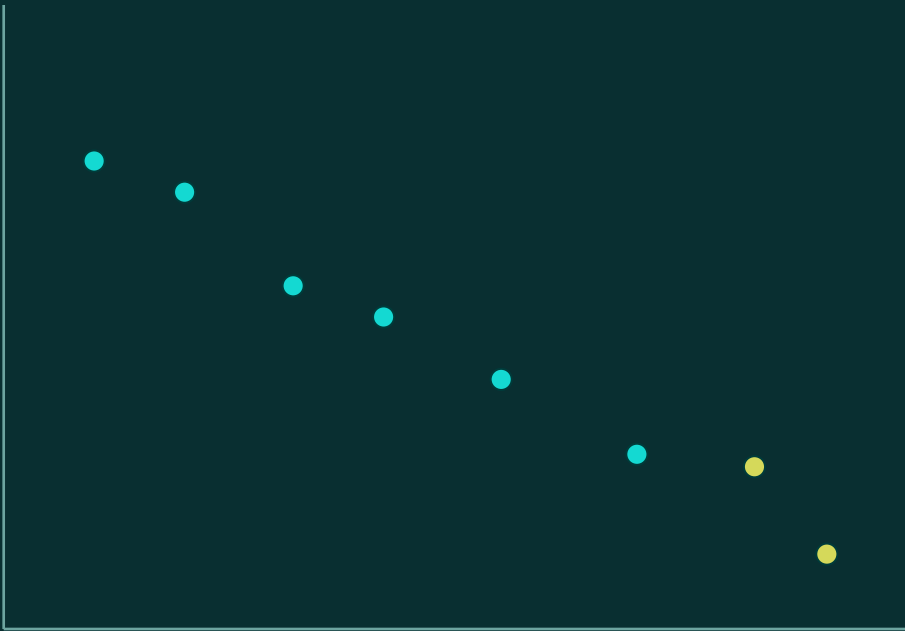
Try bins=5 and bins=20. Bin choice changes the level of detail, not the data.

Scatter and Box Plots

Use them for relationships and distributions across categories

Scatter plot

Do errors relate to completion?



```
axes = records.plot.scatter(  
    x="error_count",  
    y="completion",  
    figsize=(8, 5),  
)  
  
axes.set_title(  
    "Errors Compared with Completion"  
)  
  
plt.show()
```

```
axes = records.boxplot(  
    column="completion",  
    by="source",  
    figsize=(8, 5),  
)  
  
plt.suptitle("")  
plt.show()
```

A scatter plot can show association. It does not prove causation.

Make Static Charts Readable and Shareable

Figure size, label rotation, layout, and saving images

```
axes = status_counts.plot(
    kind="bar",
    figsize=(8, 5),
)

axes.set_title("Records by Processing Status")
axes.set_xlabel("Processing Status")
axes.set_ylabel("Number of Records")

plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
figure = axes.get_figure()

figure.savefig(
    "total_records_by_source.png",
    bbox_inches="tight",
)
```

When to save

Save a PNG when the chart needs to be included in a report, slide deck, or separate handout.

Avoid Misleading or Confusing Charts

Honest chart choices are part of the analysis

Pie chart for everything

many categories are hard to compare

3D effects

distort apparent sizes

Unsorted time

dates must be chronological

Metric ambiguity

row count $\neq$ record volume

Decoration overload

emphasize data, not effects

Correlation claims

association does not prove cause

The chart should help answer the question honestly and clearly.

Guided Exercise Part 1: Load, Inspect, and Compare Sources

Students use `block_42_equipment_activity.csv` or cleaned output from Block 44

```
import pandas as pd
import matplotlib.pyplot as plt

records = pd.read_csv(
    "block_42_equipment_activity.csv"
)

records["processed_date"] = pd.to_datetime(
    records["processed_date"]
)

records.head()
records.info()
```

Tasks

1. Count rows by source with `value_counts()`
2. Sort values
3. Plot horizontal bars
4. Add title and axis labels
5. Then calculate total `record_count` by source
6. Explain why counts and totals can differ

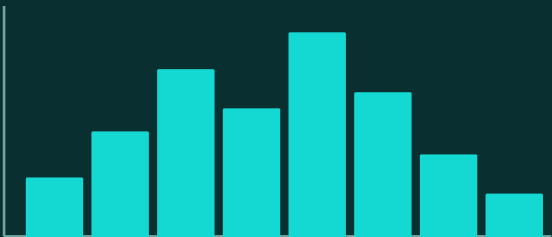
Time box: 10 minutes

Guided Exercise Part 2: Distribution, Time, and Relationship

Build three more charts from prepared pandas data

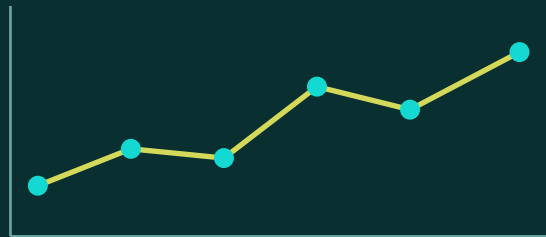
Completion distribution

Histogram of completion;
compare bins=5 and
bins=15



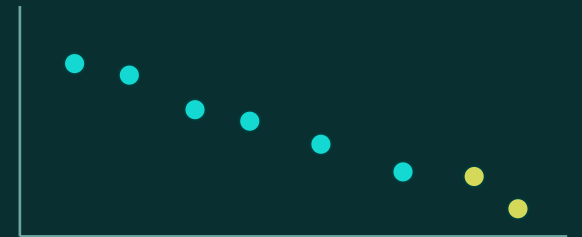
Weekly volume

Set processed_date index;
resample by week; sum
record_count



Errors vs completion

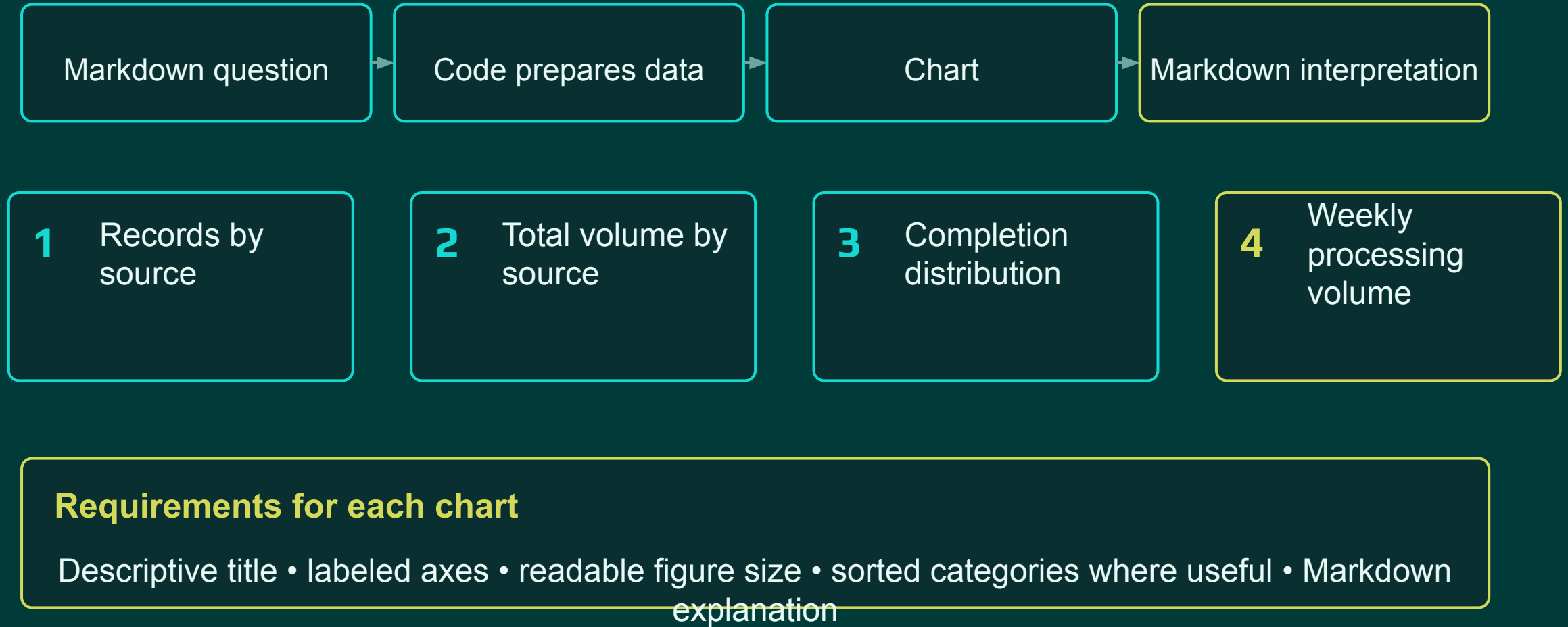
Scatter plot with
error_count on x and
completion on y



For each chart, write one observation and one conclusion the chart does not prove.

Applied Challenge: Static Operational Overview

Create a small notebook report, not a disconnected pile of charts



Knowledge Check and Wrap-Up

Students should be ready to create static notebook visualizations

Why should a chart begin with a question?

Why should dates be parsed before time charts?

When should you use a bar chart, line chart, histogram, or scatter plot?

What does `plt.tight_layout()` help prevent?

How are row counts different from summed `record_count`?

What should a Markdown interpretation say after a chart?

Final idea: the analysis includes the question, preparation, calculation, visualization, and explanation.